Project Proposal: SSH Tunneling System

Hüseyin Kaya

Student ID: 220303036

Date: 24/11/2025

Abstract

The goal of this project is to create a simple SSH-style secure tunnel between a client and a server. The system uses **TCP sockets** for the connection and performs a **key exchange** with **RSA** to create a shared **AES session key**. All data is encrypted with AES before it is sent through the tunnel.

The client can send both text messages and real files, such as images or PDF documents. Each file is divided into smaller parts called **frames**. Every frame includes a small header and an encrypted data block. To simulate transmission problems, **random noise** may be added to some frames. A **CRC-based error check** is used to detect corrupted frames.

The server receives all frames, decrypts them with the session key, checks for errors, and rebuilds the original file. The whole system will be implemented in the **C programming language** using basic socket functions and standard cryptographic operations.

Arel Üniversitesi, Müh. Mim. Fak. Bilgisayar Mühenendisliği. Türkoba Mah., Erguvan Sok., No:26, K 34537, Tepekent – Büyükçekmece, Istanbul

1 Introduction

This project implements a simple SSH-style encrypted tunnel between a client and a server.The tunnel is created between a client and a server using **TCP socket programming**. After the TCP connection is established, the client and server perform an **RSA key exchange**. A random **AES session key** is then generated and used to encrypt all data sent through the tunnel.

The client can send two main types of data: normal text messages and real files such as images or PDF files. Before transmission, each file is divided into fixed-size **frames**. Every frame contains a **frame ID**, **payload size**, a **CRC value**, and an **AES-encrypted payload**. These frames are transmitted sequentially from the client to the server over the encrypted tunnel.

During transmission, optional **noise injection** can modify some bits inside a frame to simulate real network errors. The server uses the CRC value to detect which frames are corrupted. If necessary, the client can resend these frames. When all valid frames arrive, the server decrypts the data and reconstructs the original file.

The implementation will be done in the **C programming language** using standard socket functions and cryptographic routines for RSA and AES.

2 Theory

This section explains the fundamental concepts used in the SSH-style tunneling system. The tunnel relies on reliable TCP communication, RSA-based key exchange, AES symmetric encryption for data confidentiality, and CRC mechanisms for error detection. Noise injection is also used to simulate transmission errors and evaluate the robustness of the protocol.

2.1 TCP Socket Communication

TCP provides reliable and ordered communication between the client and server. The client initiates the connection using `connect()`, while the server uses `bind()`, `listen()`, and `accept()`. Data transfer is performed through `send()` and `recv()`.

2.2 RSA Key Exchange

RSA enables secure asymmetric key exchange. The server sends its public key to the client, which encrypts a newly generated AES key using this public key. Only the server can decrypt the AES key using its private key.

2.3 AES Symmetric Encryption

AES is used for encrypting all file payloads and text messages. Both the client and the server use the same AES session key to quickly encrypt and decrypt the data.

2.4 CRC Error Detection

CRC adds an integrity checksum to every frame. When the server receives a frame, it recomputes the CRC. If the computed value does not match the received CRC, the frame is considered corrupted.

2.5 Noise Injection

Noise injection simulates transmission errors by flipping random bits in selected frames. This technique helps test the error detection and recovery capabilities of the system.

3 System Design and Diagrams

3.1 UML Class Diagram

The following UML Class Diagram illustrates the main structural components of the tunneling system. It shows how the Client, Server, Tunnel, and Frame classes interact with each other during connection setup, key exchange, encrypted data transfer, and file reconstruction.

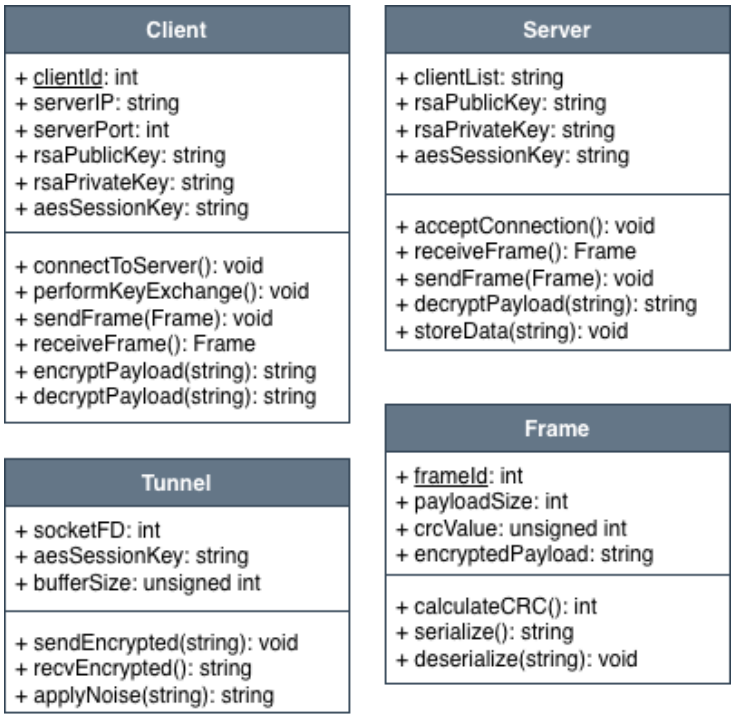

Figure 1: UML Class Diagram for the SSH Tunneling System

3.2 Flowcharts

This subsection includes two flowcharts that describe how the system establishes the tunnel and transfers files.

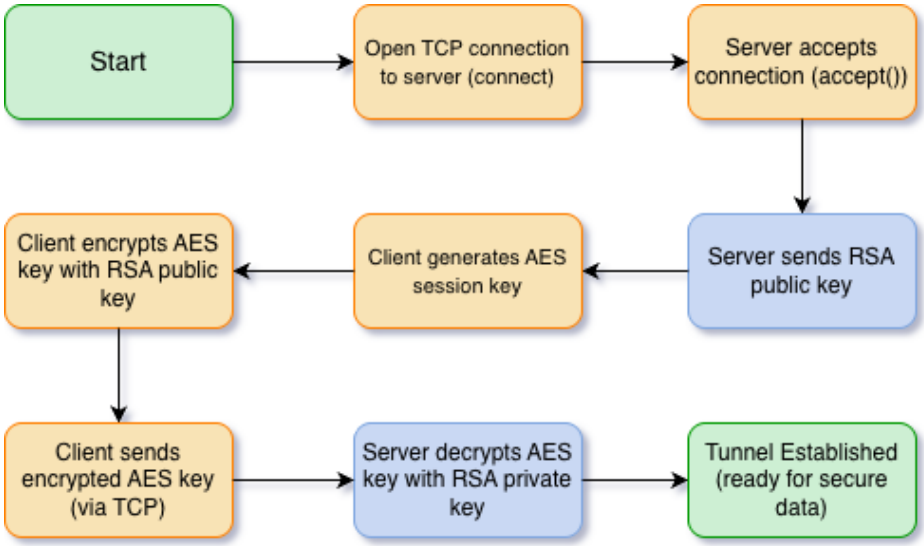

Figure 2: Flowchart for Establishing the SSH Tunnel

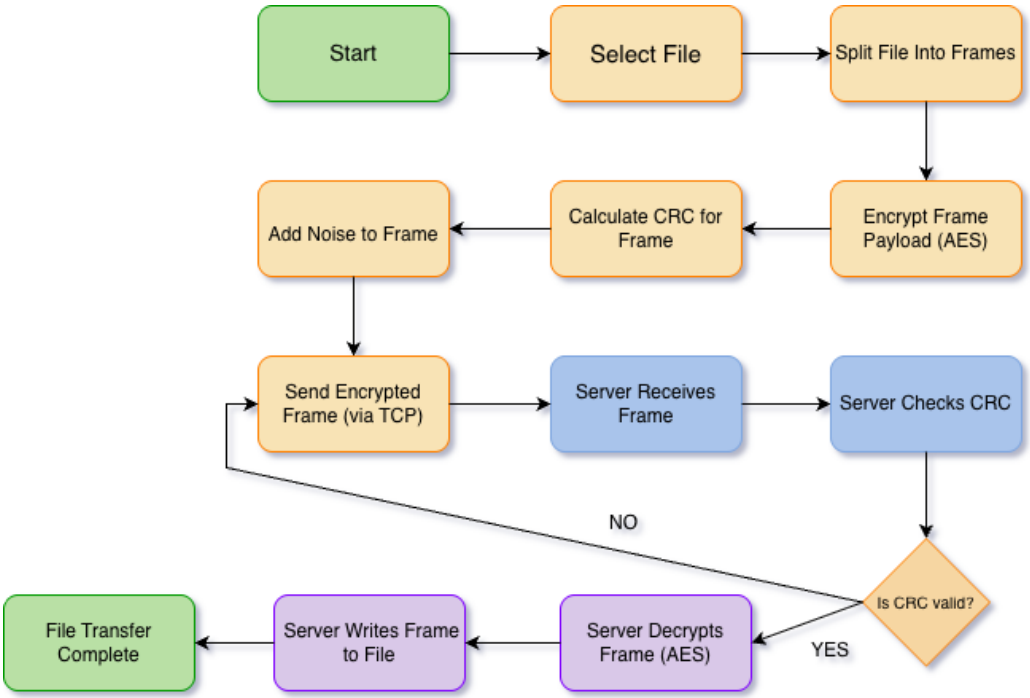

Figure 3: Flowchart for Sending a File Through the Tunnel

References

1. William Stallings, *Data and Computer Communications*, 10th ed., Pearson, 2014.

2. Daniel J. Barrett and Richard E. Silverman, *SSH, The Secure Shell: The Definitive Guide*, O'Reilly Media, 2005.

3. SSH Academy. (n.d.). *SSH Tunneling Explained*. Retrieved from `https://www.ssh.com/academy/ssh/tunneling`

4. Gökhan Şengün. (2020). *SSH Tunneling Nedir ve Nerelerde Kullanılır?*. Medium. Retrieved from `https://medium.com/@gokhansengun/ssh-tunneling-nedir-ve-nerelerde-kullan%C4%B1l%C4%B1r-2e2377c3e861`

5. Hacking Articles. (n.d.). *Comprehensive Guide on SSH Tunneling*. Retrieved from `https://www.hackingarticles.in/comprehensive-guide-on-ssh-tunn`